Estrategia de Arquitectura Evolutiva y Plan de Implementación Asistido por IA para la Plataforma ABUVI

1. Visión Estratégica: El Renacimiento del Arquitecto de Software en la Era de la IA

La transición profesional de un desarrollador senior con más de quince años de experiencia en el ecosistema Microsoft (.NET, SQL Server) y tecnologías web fundacionales (AngularJS, jQuery) hacia el panorama actual del desarrollo de software no debe interpretarse como un reinicio, sino como una evolución estratégica de capacidades. La propuesta para la plataforma ABUVI no es simplemente un ejercicio de codificación; representa la convergencia de una sólida madurez arquitectónica con las nuevas metodologías de "Desarrollo Agéntico".

En el contexto actual, herramientas como Google Antigravity, Cursor y Windsurf han desplazado el paradigma del autocompletado de código (como GitHub Copilot en sus inicios) hacia la orquestación de agentes autónomos. Para un perfil con su trayectoria, esto significa que el rol cambia de "programador que escribe sintaxis" a "arquitecto que dirige sistemas". Su experiencia en el diseño de bases de datos relacionales y la lógica de negocio en C# es el activo más valioso para guiar a estos modelos de lenguaje (LLMs), que, aunque capaces de generar código, carecen de la visión sistémica y el contexto histórico que un desarrollador senior posee.[1]

Este informe técnico detalla una hoja de ruta exhaustiva para la construcción de la aplicación web de ABUVI. Se ha diseñado específicamente para capitalizar su memoria muscular en C# y SQL, integrar sus nuevas competencias en Python para Ciencia de Datos, y adoptar frameworks de frontend modernos que resuenan conceptualmente con su experiencia en AngularJS, todo ello orquestado mediante un flujo de trabajo agéntico de vanguardia.

1.1 El Cambio de Paradigma: De Capas Horizontales a Cortes Verticales

Una de las preguntas fundamentales planteadas para la estructuración del desarrollo con IA es la dicotomía entre separar el proyecto por capas (Base de Datos, Backend, Frontend) o por funcionalidades. La respuesta, respaldada por la eficiencia de los modelos de IA actuales, es inequívoca: **la arquitectura debe orientarse a Cortes Verticales (Vertical Slices)**.

Históricamente, el desarrollo en.NET se basaba en la Arquitectura en Capas (N-Tier): Controladores, Servicios, Repositorios y Entidades separados en proyectos o carpetas distintas. Si bien esto funcionaba para equipos humanos grandes en 2010, para un

"AI-Assisted Developer" en 2026, la arquitectura en capas introduce fricción. Cuando se solicita a un agente de IA que "cree la funcionalidad de inscripción", si el contexto está disperso en cinco archivos ubicados en carpetas distantes, la probabilidad de alucinaciones o errores de integración aumenta exponencialmente debido a la fragmentación del contexto.[3]

La Arquitectura de Cortes Verticales agrupa todo lo necesario para una sola funcionalidad (API, lógica de negocio, acceso a datos y modelos) en un único "slice" o carpeta. Esto permite que herramientas como Google Antigravity carguen el contexto completo de una funcionalidad específica en la ventana de atención del modelo, resultando en una generación de código más precisa, cohesiva y funcional desde el primer intento. Por tanto, la estrategia de prompts no debe ser "créame la capa de datos", sino "implemente el corte vertical de Gestión de Inscripciones".

---

2. Selección y Justificación del Stack Tecnológico Híbrido

Para satisfacer los requisitos de eficiencia, modernización y aprovechamiento de sus habilidades existentes, se ha seleccionado una pila tecnológica que equilibra la robustez empresarial de Microsoft con la agilidad del desarrollo web moderno y la potencia analítica de Python.

2.1 Backend: La Potencia de.NET 9 y Minimal APIs

El núcleo del sistema residirá en **.NET 9**, utilizando el modelo de **Minimal APIs**. Para un veterano de ASP.NET MVC o WebAPI, la transición a Minimal APIs puede parecer inicialmente un paso hacia lo "scripting", pero en realidad es una evolución hacia la reducción de la complejidad innecesaria (boilerplate).

Característica	ASP.NET MVC / WebAPI (Tradicional)	.NET 9 Minimal APIs (Recomendado)	Impacto en Desarrollo con IA
Estructura	Controladores pesados heredando de ControllerBase.	Endpoints definidos directamente en Program.cs o módulos ligeros.	Menos tokens de código repetitivo; la IA se centra en la lógica real.
Inyección de Dependencias	Constructor Injection en cada	Inyección directa en la firma del	Simplifica la generación de

	clase Controller.	método lambda.	prompts y reduce archivos.
Rendimiento	Overhead del pipeline MVC completo.	Pipeline optimizado, menor consumo de memoria.	Ideal para despliegues en contenedores ligeros (Docker).
Sintaxis	Verbosa, requiere múltiples archivos por endpoint.	Concisa, permite definir DTOs y lógica en el mismo archivo si se desea.	Facilita la comprensión del contexto completo por parte del LLM.[5]

El uso de.NET 9 permite mantener el tipado fuerte y la seguridad en tiempo de compilación de C#, características críticas para un sistema que manejará pagos y datos de menores. Además, las mejoras en Entity Framework Core 9 facilitan la migración de su mentalidad SQL: puede escribir consultas LINQ complejas o incluso SQL crudo si es necesario, y la IA se encargará de mapear los resultados a objetos fuertemente tipados.[7]

2.2 Integración de Python: CSnakes y la Sinergia Data Science

Su reciente incursión en Python (Pandas, Numpy) no debe quedar aislada como una habilidad separada. Tradicionalmente, integrar Python en una aplicación.NET implicaba crear microservicios (por ejemplo, una API Flask) y comunicarse vía HTTP, lo cual añade latencia, puntos de fallo y complejidad operativa.

La solución estratégica para ABUVI es **CSnakes**. Esta tecnología permite integrar el tiempo de ejecución de CPython directamente dentro del proceso.NET 9. Esto significa que puede escribir scripts de análisis de datos en Python (para analizar tendencias de inscripciones, predecir ocupación de campamentos o procesar encuestas del 50 aniversario) y llamarlos desde C# como si fueran librerías nativas, sin la sobrecarga de serialización JSON ni llamadas de red.[9]

Esta integración le permite actuar como un verdadero "Full Stack Data Developer":

1. Utiliza C# para la lógica transaccional robusta (Inscripciones, Pagos).
2. Utiliza Python para la lógica analítica avanzada (Estadísticas, Recomendaciones).
3. Todo reside en un único monolito modular fácil de desplegar y mantener.

2.3 Frontend: Vue.js 3 y la Conexión con AngularJS

La elección del framework de frontend es crítica para evitar la "fatiga de JavaScript". Si bien React es popular, su curva de aprendizaje (JSX, Hooks, gestión de estado compleja) es empinada y conceptualmente distante de AngularJS. Blazor, aunque tentador por usar C#,

puede resultar pesado (WebAssembly) o requerir conexión constante (Server), lo cual no es ideal para una web pública de una asociación que debe funcionar bien en móviles con conexiones variables.[12]

Vue.js 3 es la elección óptima por su alineación filosófica con AngularJS:

- **Separación de Conceptos:** Al igual que en Angular, Vue separa claramente la plantilla HTML (<template>) de la lógica (<script>) y los estilos (<style>).
- **Directivas:** Si recuerda ng-if y ng-repeat, ya sabe usar v-if y v-for. La transferencia de conocimiento es casi inmediata.
- **Composition API:** Esta es la evolución moderna. En lugar de organizar el código por "tipos" (data, methods, computed), se organiza por "funcionalidad lógica", similar a como estructuraría un servicio en el backend. Esto resuena fuertemente con la mentalidad de un desarrollador senior de backend que valora la encapsulación y la cohesión.[13]

Para la interfaz de usuario (UI), se recomienda utilizar **PrimeVue** o **Tailwind CSS**. Dado que mencionó que no le importa pagar por componentes, PrimeVue ofrece una suite completa (calendarios, tablas de datos, galerías) que le ahorrará cientos de horas de diseño CSS manual, permitiéndole centrarse en la integración con el backend.

---

3. Entorno de Desarrollo: Orquestación con Google Antigravity

El entorno de desarrollo integrado (IDE) ha dejado de ser un simple editor de texto para convertirse en una "Misión de Control". Google Antigravity, seleccionado como su herramienta principal, ofrece capacidades que van más allá del autocompletado de GitHub Copilot o la edición rápida de Cursor. Su fortaleza radica en la gestión de agentes autónomos que pueden planificar, ejecutar y verificar tareas complejas.[1]

3.1 Configuración de la "Misión de Control"

Antigravity opera sobre una bifurcación de VS Code, lo que garantiza que sus atajos de teclado y extensiones favoritas sigan funcionando. Sin embargo, la diferencia fundamental es el **Panel de Agentes**. Aquí no solo se "chatea" con la IA; se le asignan misiones.

Para el proyecto ABUVI, configuraremos Antigravity para trabajar en **Modo de Planificación (Planning Mode)**. A diferencia del modo rápido, el modo de planificación obliga al agente a generar un "Plan de Implementación" y una "Lista de Tareas" antes de escribir una sola línea de código. Esto es crucial para un arquitecto de software: le permite revisar la estrategia del agente (por ejemplo, confirmar el esquema de base de datos propuesto) antes de autorizar la ejecución, manteniendo usted el control total del diseño del sistema.[2]

3.2 Definición de Habilidades Personalizadas (.agent Skills)

Para que la IA actúe como un desarrollador senior y no como un junior copiando de StackOverflow, debemos instruirla explícitamente sobre nuestros estándares. Antigravity permite definir "Habilidades" (Skills) en una carpeta .agent/skills dentro del proyecto. Estas habilidades actúan como directrices inmutables que el agente consultará antes de realizar tareas.[15]

A continuación, se detalla la estructura y contenido de las habilidades críticas que definiremos para ABUVI:

Habilidad 1: Arquitectura de Backend Moderno (.agent/skills/backend-arch/SKILL.md)

Esta definición asegura que el agente respete la arquitectura de cortes verticales y las prácticas modernas de.NET 9.

Habilidad: Arquitectura.NET 9 Vertical Slice

Objetivo: Generar código backend que cumpla estrictamente con los patrones de diseño de ABUVI.

Reglas de Oro:

1. **Cortes Verticales:** Cada funcionalidad (ej. RegisterCamper) debe residir en su propia carpeta dentro de Features/. Esta carpeta debe contener el Endpoint, el DTO de Request/Response, y la lógica de negocio.
2. **Minimal APIs:** NO utilizar Controladores (ControllerBase). Utilizar app.MapPost y app.MapGet con inyección de dependencias en los métodos.
3. **Inmutabilidad:** Utilizar record para todos los DTOs y mensajes internos.
4. **Validación:** Implementar validaciones utilizando FluentValidation antes de procesar cualquier lógica.
5. **Logging:** Utilizar Serilog estructurado. Nunca usar Console.WriteLine.
6. **Base de Datos:** Acceso a datos exclusivamente a través de Entity Framework Core. No usar ADO.NET crudo a menos que se especifique para optimización masiva.

Ejemplo de Estructura de Archivo Esperada:
Features/Campers/Register/RegisterCamperEndpoint.cs

Habilidad 2: Frontend Vue 3 Composition (.agent/skills/frontend-vue/SKILL.md)

Esta habilidad garantiza que el frontend sea consistente y evite mezclar estilos antiguos con modernos.

Habilidad: Desarrollo Frontend Vue 3

Objetivo: Generar componentes Vue.js modernos, limpios y tipados.

Reglas de Oro:

1. **Sintaxis:** Utilizar exclusivamente <script setup lang="ts">.
2. **Estado:** Preferir ref para primitivos y reactive para objetos complejos de formulario.
3. **Estilos:** Utilizar clases de utilidad de Tailwind CSS. No escribir bloques <style> personalizados a menos que sea para animaciones complejas.
4. **Componentes UI:** Utilizar componentes de PrimeVue para elementos interactivos (Tablas, Diálogos, DatePickers).
5. **API:** Las llamadas al backend deben encapsularse en "composables" (ej. useCampers.ts) y nunca realizarse directamente dentro del componente visual.

---

4. Biblioteca de Prompts de Ingeniería (Prompt Engineering Library)

Como "AI Orchestrator", su principal herramienta de trabajo serán los prompts estructurados. A continuación, presento una biblioteca de prompts diseñada específicamente para las fases del proyecto ABUVI, clasificados por funcionalidad y no por capas técnicas, alineándose con la arquitectura de cortes verticales.

4.1 Fase 1: Andamiaje del Proyecto (Scaffolding)

Este prompt inicial configura la estructura del repositorio, asegurando que todas las piezas (Docker,.NET, Vue) estén conectadas desde el día uno.

Prompt Maestro para Scaffolding:

> "Actúa como un Arquitecto de Software Principal especializado en soluciones Cloud-Native sobre Microsoft.NET 9.
>
> **Misión:** Inicializar la solución para el proyecto 'ABUVI Platform'.
>
> **Especificaciones Técnicas:**
>
> 1. **Estructura de Solución:** Crea una solución.NET (Abuvi.sln) con dos proyectos principales:
> - Abuvi.API: Web API en.NET 9.
> - Abuvi.Web: Proyecto Vue.js 3 creado con Vite y TypeScript.
> 2. **Infraestructura Local:** Genera un archivo docker-compose.yml que orqueste:

    - Un contenedor PostgreSQL 16 (persistencia de datos).
    - Un contenedor pgAdmin 4 (gestión visual de BBDD).
    - Un contenedor MinIO (simulación de S3 para almacenamiento de fotos del 50 aniversario).
3. **Configuración Backend:**
    - Configura Entity Framework Core con PostgreSQL.
    - Implementa Serilog para logging a consola y archivo.
    - Habilita CORS para permitir peticiones desde localhost:5173 (Vite).
4. **Configuración Frontend:**
    - Inicializa Tailwind CSS y PrimeVue.
    - Configura Axios con un interceptor base apuntando a la API.

Entregable: Ejecuta los comandos de creación, genera los archivos de configuración y entrégame un script init.ps1 que levante todo el entorno con un solo clic. Verifica que los puertos no entren en conflicto."

4.2 Fase 2: Desarrollo del Núcleo - Gestión de Inscripciones

Aquí aplicamos la arquitectura de cortes verticales. En lugar de pedir "crea la tabla de inscripciones", pedimos la funcionalidad completa.

Prompt Funcional: Inscripción de Campistas:

"Actúa como un Desarrollador Full Stack Senior. Vamos a implementar la funcionalidad crítica de 'Inscripción de Campistas' siguiendo la Arquitectura de Corte Vertical.

Requerimientos de Negocio:
Un usuario debe poder inscribir a un niño en un campamento específico. El sistema debe validar la edad del niño frente a las reglas del campamento y calcular el precio automáticamente.
Instrucciones de Implementación:

1. **Backend (Corte Vertical Features/Inscripciones):**
    - Crea un Record DTO InscripcionRequest (IdCampamento, DatosNiño, EsSocio).
    - Implementa un endpoint POST /api/inscripciones.
    - Incluye lógica de validación (FluentValidation): El niño debe tener entre 6 y 17 años.
    - Calcula el precio: Si EsSocio es true, aplica un 20% de descuento sobre el precio base del campamento.
    - Guarda la entidad en BBDD y devuelve el ID de inscripción.
2. **Frontend (Vue Component):**
    - Crea un formulario paso a paso (Wizard) usando PrimeVue.
    - Paso 1: Selección de Campamento.

    - Paso 2: Datos del Participante.
    - Paso 3: Resumen y Precio Calculado.
    - Conecta con el endpoint creado.

  Nota de Seguridad: Asegúrate de que los datos médicos (alergias) se marquen para ser almacenados en una columna cifrada o tratada con especial sensibilidad en el modelo de datos."

4.3 Fase 3: Módulo del 50 Aniversario y Archivo Histórico

Este módulo requiere manejo de archivos e interfaces visuales ricas. Aquí es donde su preocupación por la complejidad del frontend se mitiga delegando el diseño a la IA y usando componentes pre-construidos.

Prompt Funcional: Galería Histórica:

"Necesito crear el 'Muro de Recuerdos' para el 50 Aniversario.

Backend:

- Genera un servicio BlobStorageService que implemente una interfaz genérica IFileStorage.
- Implementa la versión concreta usando el SDK de MinIO (compatible con S3) para desarrollo local.
- Crea un endpoint que permita subir fotos, redimensionarlas automáticamente a 1080p usando la librería ImageSharp para ahorrar espacio, y guardar los metadatos (año, descripción, uploader) en Postgres.

Frontend:

- Diseña una galería tipo 'Masonry' (muro de ladrillos) usando CSS Grid o un componente de PrimeVue.
- Implementa 'Infinite Scroll' para cargar fotos a medida que el usuario baja.
- Añade un filtro por 'Década' (70s, 80s, 90s...).

Estilo: El diseño debe evocar nostalgia. Sugiere una paleta de colores cálida/sepia para este módulo específico dentro de Tailwind."

4.4 Fase 4: Integración de Python (Data Science)

Para aprovechar su nueva habilidad, crearemos un módulo de análisis que prediga la asistencia futura basándose en datos históricos.

Prompt de Integración CSnakes:

"Actúa como experto en Interoperabilidad.NET/Python.

Contexto: Tengo un script de Python prediccion_asistencia.py que usa Pandas y

Scikit-Learn para predecir la ocupación basándose en datos de los últimos 15 años.

Misión: Integrar este script en el backend.NET usando la librería **CSnakes**.

1. Configura el proyecto.NET para incluir las referencias a CSnakes.
2. Crea un servicio en C# AnalisisService.cs que inyecte el entorno de Python.
3. Mapea la función de Python predecir_ocupacion(dataframe) a un método C#.
4. Muestra cómo transformar una List<Inscripcion> de C# a un DataFrame de Pandas dentro del proceso de interoperabilidad.
5. Expón el resultado en un endpoint GET /api/admin/dashboard/predicciones para que la directiva pueda ver las estimaciones."

4.5 Fase 5: Refactorización y Migración de Legado

Dado que mencionó tener código antiguo, es probable que quiera reutilizar lógica de negocio compleja (ej. reglas de asignación de cabañas) que ya tiene escrita en C# antiguo o incluso lógica de frontend de AngularJS.

Prompt de Refactorización (AngularJS a Vue 3):

"Tengo el siguiente controlador de AngularJS (v1.6) que gestiona la lógica de asignación de tiendas de campaña.

Tarea: Reescribe esta lógica utilizando **Vue 3 Composition API** y **TypeScript**.

- Transforma las variables $scope en ref o reactive.
- Convierte las funciones $scope.calcular en funciones puras o composables.
- Elimina la dependencia de $http y utiliza la instancia de Axios configurada en el proyecto.
- Mantén la lógica de negocio intacta pero adáptala a los tipos estrictos de TypeScript."

---

5. Profundización Técnica y Mejores Prácticas

5.1 Base de Datos: Estrategia Relacional Evolutiva

Para un experto en SQL, el cambio a Entity Framework (EF) Core puede generar desconfianza sobre el rendimiento de las consultas generadas. Sin embargo, en.NET 9, EF Core es extremadamente eficiente.

- **Recomendación:** Utilice el enfoque "Code-First" pero con supervisión. Pida al agente: *"Genera la migración para añadir la tabla Campamentos, pero muéstrame el SQL resultante antes de aplicarlo"*. Esto le permite validar índices, claves foráneas y tipos de

datos (ej. asegurar que usa decimal para dinero y no float) con su ojo experto.[17]

- **Mapas:** Para el mapa interactivo de campamentos, almacene las coordenadas como tipo geography en PostGIS (extensión de Postgres) o simplemente como lat/long (double) si las consultas geoespaciales son simples. El frontend usará **Leaflet.js**, que es ligero y fácil de encapsular en un componente Vue.[18]

5.2 Pagos: Seguridad y Redsys (Banco Sabadell)

La gestión de pagos es delicada. No almacene números de tarjeta. En lugar de una solución de pago integrada como Stripe, la estrategia se enfocará en la integración directa con la pasarela de Redsys (Banco Sabadell), común en el contexto español.

- Enfoque de Integración: Investigar librerías y paquetes NuGet para la integración de la pasarela de pagos 'Redsys' (Banco Sabadell) en .NET 9, enfocándose en la generación de firmas SHA-256 y la gestión de notificaciones asíncronas (Webhooks) seguras.
- Flujo: El frontend solicita al backend los datos necesarios para generar el formulario de pago. El backend genera la firma SHA-256 con los parámetros de la transacción y redirige al usuario al TPV virtual de Redsys. Tras el pago, Redsys notifica el resultado al backend mediante un Webhook (comunicación asíncrona).
- Prompt de Webhook (Redsys): *"Crea un endpoint seguro para recibir las notificaciones asíncronas (Webhooks) de Redsys. La lógica debe incluir la verificación estricta de la firma SHA-256 del mensaje para garantizar su autenticidad. Si el estado de la transacción es 'Pago Aceptado', actualiza el estado de la inscripción a 'Pagada' en la base de datos de forma transaccional, asegurando la robustez contra reintentos o fallos en la comunicación."*.

5.3 Despliegue: "Low-Ops" para Asociaciones

Para ABUVI, evitar costes recurrentes altos es clave.

- **Contenedores:** Empaquete el Backend y el Frontend en imágenes Docker.
- **Orquestación Simple:** Use Docker Compose en un VPS Linux económico (ej. Hetzner, 5€/mes).
- **Proxy Inverso:** Configure **Caddy Server** o **Traefik** como proxy inverso. Estos gestionan automáticamente los certificados SSL (HTTPS) de Let's Encrypt, liberándole de la renovación manual de certificados, un dolor de cabeza común en la era antigua de IIS.[22]

---

6. Conclusión y Siguientes Pasos

Su perfil técnico es ideal para liderar este proyecto. No necesita "aprender a programar" de nuevo; necesita aprender a "dirigir a quien programa" (la IA). La elección de **.NET 9 + Vue 3** respeta su herencia técnica mientras moderniza la entrega. La incorporación de **CSnakes** valida y potencia su inversión en Python, y el uso de **Google Antigravity** le otorga el

superpoder de multiplicar su productividad.

Pregunta Técnica para Profundizar:
Dado que el archivo histórico del 50 aniversario contendrá muchas imágenes escaneadas antiguas, ¿desea que incorporemos en la fase de "Ingesta de Datos" un paso de procesamiento con IA (usando una librería Python en CSnakes) para restaurar automáticamente el color o mejorar la resolución de las fotos antes de guardarlas en el Blob Storage? Esto añadiría un valor inmenso y tangible para los socios.

Recomendación Final de Inicio

No intente escribir todo el código base hoy. Abra Antigravity, configure la carpeta .agent con las habilidades descritas, y lance el primer prompt de Scaffolding. Deje que la IA construya los cimientos mientras usted revisa los planos.

Resumen de Citaciones:
1 - Metodología Agéntica y Antigravity.
5 -.NET 9 y Minimal APIs.
9 - Interoperabilidad Python/NET (CSnakes).
12 - Comparativa Frontend (Vue vs Blazor/Angular).
3 - Arquitectura de Cortes Verticales.
18 - Mapas Interactivos.
20 - Integración de Pagos Stripe.

Obras citadas

1. Build with Google Antigravity, our new agentic development platform, fecha de acceso: enero 19, 2026, https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/
2. Getting Started with Google Antigravity, fecha de acceso: enero 19, 2026, https://codelabs.developers.google.com/getting-started-google-antigravity
3. Is this true what a senior dev said on Linkedin about "The hidden cost of "enterprise" .NET architecture" : r/csharp - Reddit, fecha de acceso: enero 19, 2026, https://www.reddit.com/r/csharp/comments/1o3vs55/is_this_true_what_a_senior_dev_said_on_linkedin/
4. vyancharuk/nodejs-api-boilerplate: 🛠️ A production-ready LLM-Powered Node.js & TypeScript REST API template, with a focus on Clean Architecture - GitHub, fecha de acceso: enero 19, 2026, https://github.com/vyancharuk/nodejs-api-boilerplate
5. dotnet scaffold - Next Generation Content Creation for .NET - Microsoft Dev Blogs, fecha de acceso: enero 19, 2026, https://devblogs.microsoft.com/dotnet/introducing-dotnet-scaffold/
6. [Showcase] .NET 9 SaaS API Template — built to kickstart your next project - Reddit, fecha de acceso: enero 19, 2026,

https://www.reddit.com/r/dotnet/comments/1m2lx2p/showcase_net_9_saas_api_template_built_to/
7. Code bases with Modern C# in 2025 : r/csharp - Reddit, fecha de acceso: enero 19, 2026, https://www.reddit.com/r/csharp/comments/1osl8tm/code_bases_with_modern_c_in_2025/
8. AI in C# and .NET Development: Google Antigravity IDE - DEV Community, fecha de acceso: enero 19, 2026, https://dev.to/iron-software/ai-in-c-and-net-development-google-antigravity-ide-5a72
9. Embedding Python into your .NET project with CSnakes - Anthony Shaw, fecha de acceso: enero 19, 2026, https://tonybaloney.github.io/posts/embedding-python-in-dot-net-with-csnakes.html
10. Embedding Python in .NET with CSnakes | atal upadhyay - WordPress.com, fecha de acceso: enero 19, 2026, https://atalupadhyay.wordpress.com/2025/12/05/embedding-python-in-net-with-csnakes/
11. Deep .NET - Using AI Python Libraries in .NET Apps with CSnakes - YouTube, fecha de acceso: enero 19, 2026, https://www.youtube.com/watch?v=DqoxHNH9Iwo
12. Angular vs Blazor: Choosing the right framework for your web application - evoila US, fecha de acceso: enero 19, 2026, https://evoila.com/us/blog/angular-vs-blazor-choosing-right-framework-web-app/
13. Vue 3 - The Composition API (Part 1) | newline - Newline.co, fecha de acceso: enero 19, 2026, https://www.newline.co/@kchan/vue-3-the-composition-api-part-1--afbd9dbf
14. A Beginner's Guide to Vue.js Part-7: Composition API and Composables - Medium, fecha de acceso: enero 19, 2026, https://medium.com/@vasanthancomrads/a-beginners-guide-to-vue-js-part-7-composition-api-and-composables-a2288a6ea46a
15. Getting Started with Skills in Google Antigravity, fecha de acceso: enero 19, 2026, https://codelabs.developers.google.com/getting-started-with-antigravity-skills
16. Tutorial : Getting Started with Google Antigravity Skills, fecha de acceso: enero 19, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-antigravity-skills-864041811e0d
17. Technology | 2025 Stack Overflow Developer Survey, fecha de acceso: enero 19, 2026, https://survey.stackoverflow.co/2025/technology
18. Leaflet Examples - MapTiler documentation, fecha de acceso: enero 19, 2026, https://docs.maptiler.com/leaflet/examples/
19. Top 7 JavaScript Libraries for Creating Dynamic Maps - Colorlib, fecha de acceso: enero 19, 2026, https://colorlib.com/wp/javascript-libraries-for-creating-dynamic-maps/

20. How to use Stripe for nonprofits | The Jotform Blog, fecha de acceso: enero 19, 2026, https://www.jotform.com/blog/stripe-for-nonprofits/
21. Charity payment processing explained: Tips for improving the donation experience - Stripe, fecha de acceso: enero 19, 2026, https://stripe.com/ae/resources/more/charity-payment-processing-explained
22. Modern Web App Pattern for .NET - Azure Architecture Center | Microsoft Learn, fecha de acceso: enero 19, 2026, https://learn.microsoft.com/en-us/azure/architecture/web-apps/guides/enterprise-app-patterns/modern-web-app/dotnet/guidance